



JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

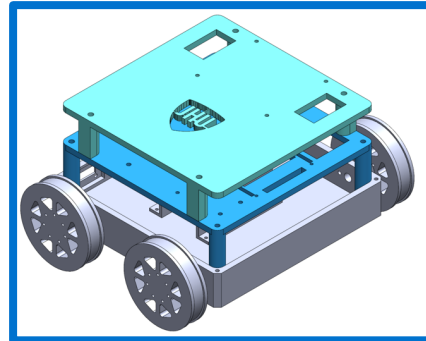
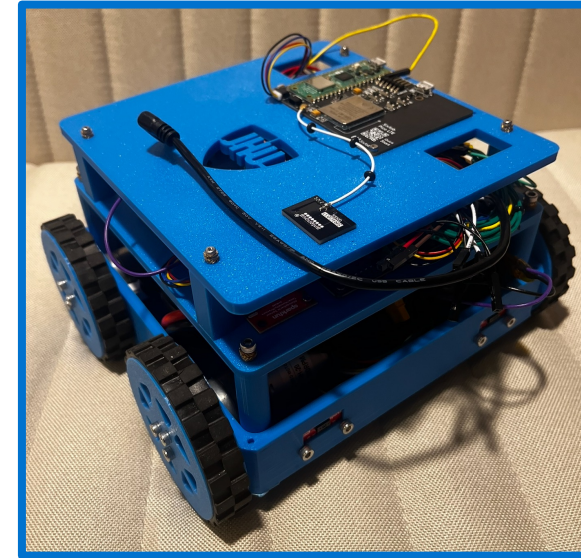
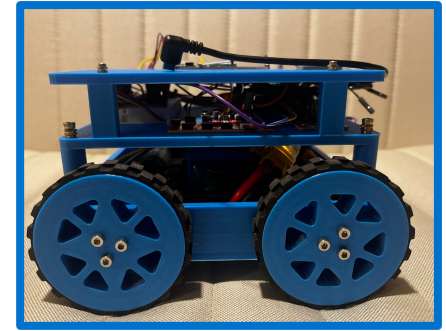
JH μ BOT

MicroMachineLearning-**Micro**controller-**Micro**python-I2C based Robot

Nareg Shirajian

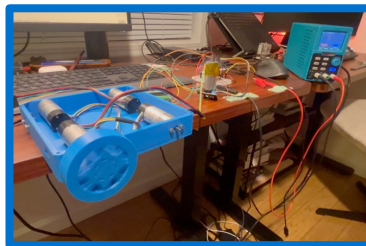
What is JH μ BOT ?

- **JH μ BOT** is a robotics kit and development platform built around a primarily 3D-printed, four-wheel rover. It is designed to support a wide range of applications, including free-roaming navigation, motion and path planning, mapping, terrain detection, and autonomous delivery.
- **Hardware Stack:** PICO 2 LTE/GPS, IMU (9 DOF), Motor Drivers, Motors, ToF Imager, Temp Sensor, External PWM Driver, Lipo Battery
- **Software Stack:** Micro python!

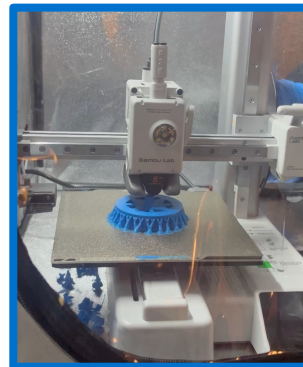


The Early Days

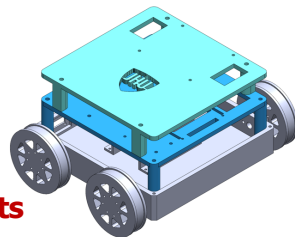
- My motivation for this project stems from a simple question: ***Is it possible, and how, to make a highly capable robot using only a microcontroller?*** capabilities of modern microcontrollers. Rather than settling for a basic motorized rover, the aim is to build a functional, practical, and low-cost platform for real research. This is achieved through efficient programming, seamless sensor integration, and fully utilizing the processing
- The robot's chassis and wheels were 3D-printed, with all sensors sourced from SparkFun and Adafruit product lines. These sensors were interconnected using an I²C daisy-chain architecture, enabling streamlined integration and offering the potential for consolidation through a single wrapper file encompassing all necessary drivers.



First ever motor test (Video)

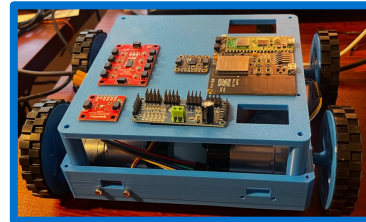
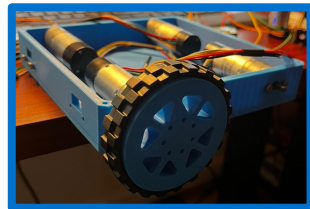


3D Printing the wheel (Video)



CAD Assembly of Printed Components

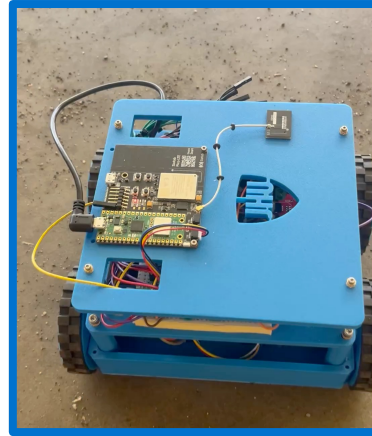
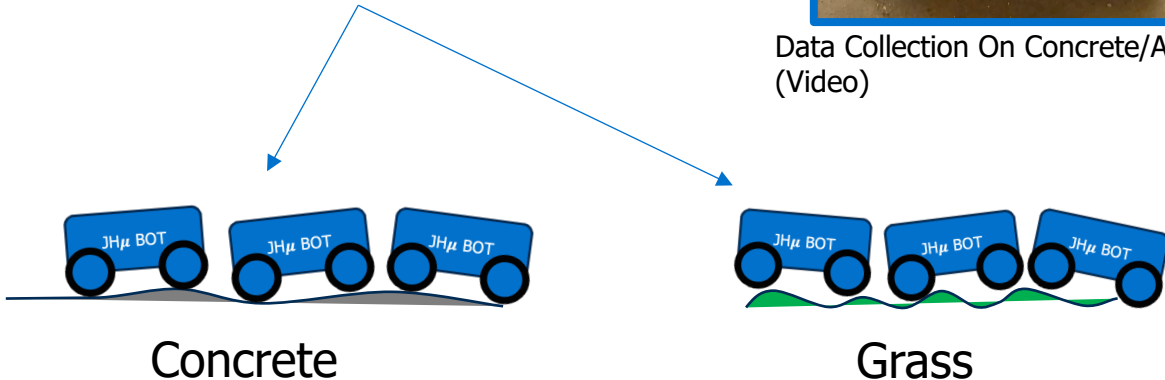
Rough Assembly Flow



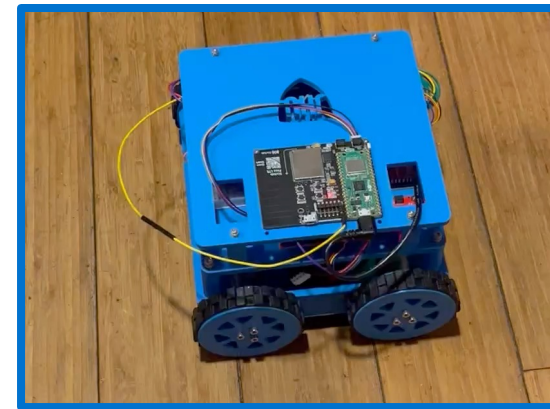
Initial Testing & Data Collection

- For the scope of this class, **I chose to focus on IMU-based terrain classification and wheel-slip detection.**

- The approach was based on the simple observation that, because the robot acts as a rigid body, **terrain irregularities are directly reflected in the IMU measurements.**



Data Collection On Concrete/Asphalt (Video)

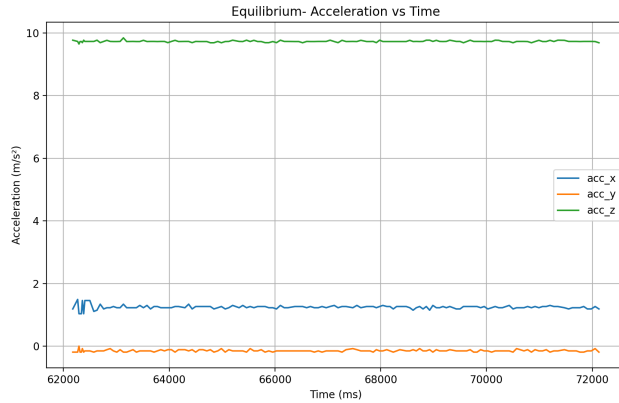


Rolling Chassis Test (Video)



Data Collection On Grass/Dirt (Video)

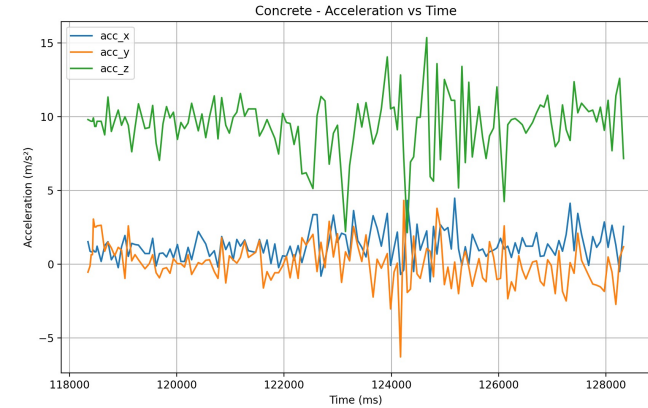
IMU Readings



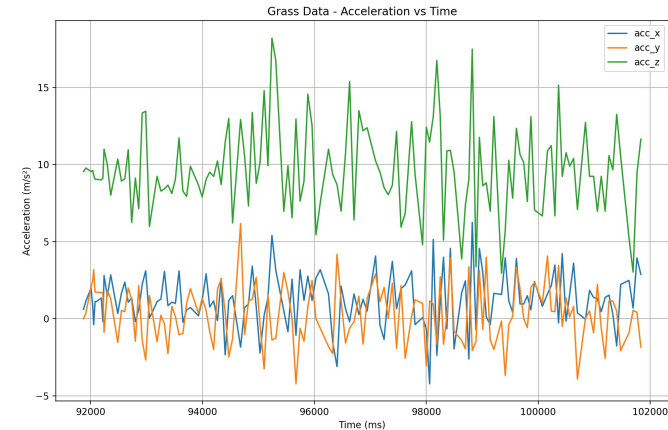
Raw Data-Equilibrium

For the equilibrium, the robot was placed on a platform such that the wheels were spinning with no load and no contact to ground

Raw Data- Concrete

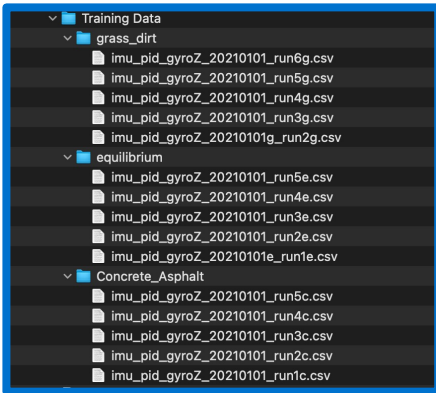


Raw Data- Grass



Deployed ML Logic on PICO (Simplified Approach)

IMU Dataset



Desktop Python ML Decision Tree
Algorithm Output (Depth = 3)

```

-- gyro_z <= 0.01
|   -- gyro_z <= -0.01
|   |   -- acc_z <= 12.85
|   |   |   class: 0
|   |   -- acc_z > 12.85
|   |   |   class: 2
|   -- gyro_z > -0.01
|   |   -- gyro_x <= 0.02
|   |   |   class: 1
|   |   -- gyro_x > 0.02
|   |   |   class: 2
-- gyro_z > 0.01
|   -- acc_x <= -0.32
|   |   -- acc_x <= -0.71
|   |   |   class: 2
|   |   -- acc_x > -0.71
|   |   |   class: 2
|   -- acc_x > -0.32
|   |   -- gyro_y <= 0.51
|   |   |   class: 0
|   |   -- gyro_y > 0.51
|   |   |   class: 2

```

Micro python Code from python derived

```

1  if gyro_z <= 0.01:
2      if gyro_z <= -0.01:
3          if acc_z <= 12.85:
4              return 0
5          else: # acc_z > 12.85
6              return 2

```

Using the “**export_text**” from Scikit-learn The decision tree outputs the model in readable logical format; we then convert into If/else code format that can be deployed onto the **JHμ BOT**

$$\hat{y} = \begin{cases} 0, & \text{if } \text{gyro_z} \leq 0.01 \wedge \text{gyro_z} \leq -0.01 \wedge \text{acc_z} \leq 12.85 \\ 2, & \text{if } \text{gyro_z} \leq 0.01 \wedge \text{gyro_z} \leq -0.01 \wedge \text{acc_z} > 12.85 \\ 1, & \text{if } \text{gyro_z} \leq 0.01 \wedge \text{gyro_z} > -0.01 \wedge \text{gyro_x} \leq 0.02 \\ 2, & \text{if } \text{gyro_z} \leq 0.01 \wedge \text{gyro_z} > -0.01 \wedge \text{gyro_x} > 0.02 \\ 2, & \text{if } \text{gyro_z} > 0.01 \wedge \text{acc_x} \leq -0.32 \wedge \text{acc_x} \leq -0.71 \\ 2, & \text{if } \text{gyro_z} > 0.01 \wedge \text{acc_x} \leq -0.32 \wedge \text{acc_x} > -0.71 \\ 0, & \text{if } \text{gyro_z} > 0.01 \wedge \text{acc_x} > -0.32 \wedge \text{gyro_y} \leq 0.51 \\ 2, & \text{if } \text{gyro_z} > 0.01 \wedge \text{acc_x} > -0.32 \wedge \text{gyro_y} > 0.51 \end{cases}$$

(Formal Notation)

Since the Pico has internal file storage, I wrote a script that makes the robot drive for 10 seconds and poll & store data from IMU. I then transferred the files to my desktop

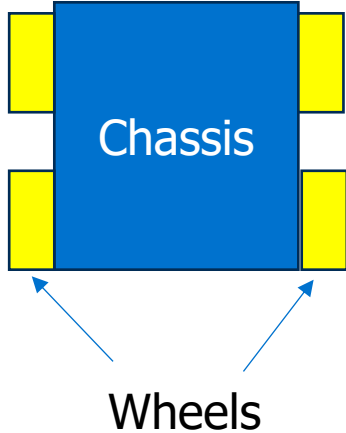
Deployed ML Logic on PICO (Results!)



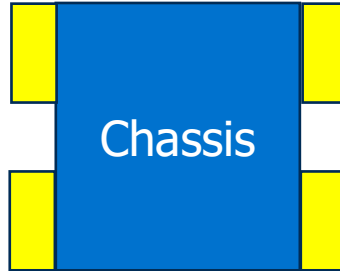
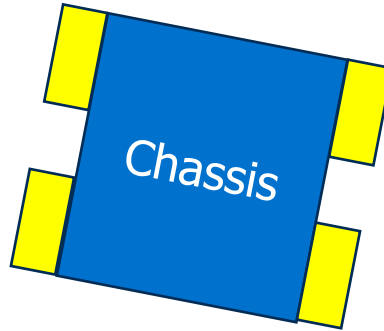
(Video)

```
time_ms,acc_x,acc_y,acc_z,gyro_x,gyro_y,gyro_z,yaw_error,predicted_label
309090,1.340,0.344,0.691,0.000,0.000,0.000,-0.000,equilibrium
309130,1.340,0.344,0.691,0.000,0.000,0.000,-0.000,equilibrium
309145,1.266,0.461,0.727,0.033,0.021,-0.025,0.025,concrete_asphalt
309168,1.266,0.461,0.727,0.033,0.021,-0.025,0.025,concrete_asphalt
309184,1.457,4.176,9.730,0.033,0.021,-0.025,0.025,concrete_asphalt
309205,1.531,2.488,9.305,0.102,0.021,0.018,-0.018,concrete_asphalt
309227,1.531,2.488,9.305,0.102,0.021,0.018,-0.018,concrete_asphalt
309278,1.074,2.680,9.199,0.203,0.082,-0.006,0.006,grass_dirt
309301,1.492,2.297,9.000,-0.125,-0.025,0.033,-0.033,concrete_asphalt
309325,1.492,2.297,9.000,-0.125,-0.025,0.033,-0.033,concrete_asphalt
309345,1.492,0.422,10.492,-0.039,-0.096,-0.076,0.076,concrete_asphalt
309365,1.570,2.645,9.426,-0.039,-0.096,-0.076,0.076,concrete_asphalt
309385,1.570,2.645,9.426,-0.039,-0.096,-0.076,0.076,concrete_asphalt
309401,1.531,0.652,8.883,0.178,0.033,0.193,-0.193,concrete_asphalt
309425,0.383,3.027,9.500,0.178,0.033,0.193,-0.193,concrete_asphalt
309447,0.383,3.027,9.500,0.002,0.098,0.080,-0.080,concrete_asphalt
309467,1.992,1.301,11.074,0.002,0.098,0.080,-0.080,concrete_asphalt
309487,0.883,0.996,9.160,0.336,0.063,-0.074,0.074,concrete_asphalt
309509,0.883,0.996,9.160,0.336,0.063,-0.074,0.074,concrete_asphalt
309524,1.723,2.262,10.914,0.336,0.063,-0.074,0.074,concrete_asphalt
309544,1.227,0.805,10.344,-0.625,0.037,0.041,-0.041,concrete_asphalt
309567,1.227,0.805,10.344,-0.625,0.037,0.041,-0.041,concrete_asphalt
309583,1.418,1.340,9.004,-0.625,0.037,0.041,-0.041,concrete_asphalt
309604,1.723,1.379,10.953,0.053,-0.082,0.063,-0.063,concrete_asphalt
309626,1.723,1.379,10.953,0.053,-0.082,0.063,-0.063,concrete_asphalt
309643,1.418,0.461,9.305,-0.111,0.090,-0.020,0.020,concrete_asphalt
309666,1.418,0.461,9.305,-0.111,0.090,-0.020,0.020,concrete_asphalt
309685,1.227,0.766,9.926,-0.111,0.090,-0.020,0.020,concrete_asphalt
309701,0.727,0.652,8.922,-0.152,-0.021,0.102,-0.102,concrete_asphalt
309725,0.727,0.652,8.922,-0.152,-0.021,0.102,-0.102,concrete_asphalt
309744,1.035,-0.383,9.887,-0.240,-0.133,0.035,-0.035,concrete_asphalt
309767,2.605,1.188,9.766,-0.240,-0.133,0.035,-0.035,concrete_asphalt
309787,2.605,1.188,9.766,-0.240,-0.133,0.035,-0.035,concrete_asphalt
309803,0.727,-0.152,9.121,0.422,-0.258,0.025,-0.025,concrete_asphalt
309826,0.574,2.105,9.344,0.422,-0.258,0.025,-0.025,concrete_asphalt
309846,0.574,2.105,9.344,0.352,0.107,0.092,-0.092,concrete_asphalt
309865,0.613,-0.113,9.082,0.352,0.107,0.092,-0.092,concrete_asphalt
309885,1.227,1.648,9.199,-0.410,0.445,-0.088,0.088,concrete_asphalt
309909,1.227,1.648,9.199,-0.410,0.445,-0.088,0.088,concrete_asphalt
309925,1.188,-0.113,10.230,-0.410,0.445,-0.088,0.088,concrete_asphalt
309944,1.379,-0.918,10.109,0.305,-0.275,-0.076,0.076,concrete_asphalt
309967,1.379,-0.918,10.109,0.305,-0.275,-0.076,0.076,concrete_asphalt
309983,1.227,0.461,9.965,0.305,-0.275,-0.076,0.076,concrete_asphalt
```


Classical Control (PID), (PI), (P)



Robot yawed to the right due to uneven terrain



The IMU measured the robot's angular velocity and applied corrective action by modulating the PWM of the right-side wheels to counteract the rotation.

PID Based control about the YAW axis

Algorithm 2 Yaw-Axis P Control Loop ($K_i = 0$, $K_d = 0$)

Require: Desired yaw rate ω_{ref} ; measured yaw rate g_z ; gain

K_p ; base PWM PWM_0

Ensure: Motor PWMs (PWM_{left} , $\text{PWM}_{\text{right}}$)

1: $e_{\text{prev}} \leftarrow 0$

2: **while** robot is active **do**

3: $e \leftarrow \omega_{\text{ref}} - g_z$

4: $u \leftarrow K_p \cdot e$ ▷ Since $K_i = 0$ and $K_d = 0$, no I or D term

5: $\text{PWM}_{\text{left}} \leftarrow \text{PWM}_0 - u$

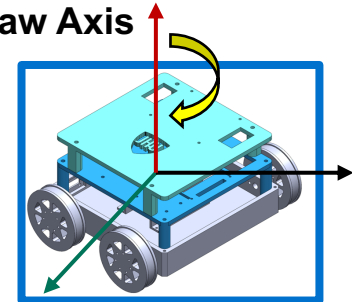
6: $\text{PWM}_{\text{right}} \leftarrow \text{PWM}_0 + u$

7: Apply PWM via PCA9685

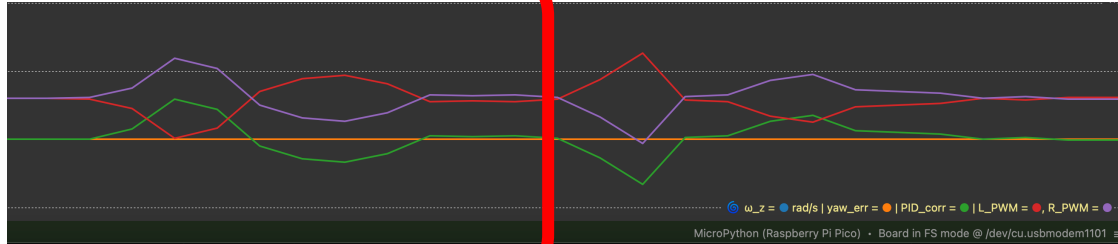
8: $e_{\text{prev}} \leftarrow e$

9: **end while**

Yaw Axis



Classical Control (PID), (PI), (P)



Observing the RED and PURPLE lines, when a change in rotational velocity is detected, the left and right sides adjust in opposite directions to counteract the motion.

PID Based control about the YAW axis

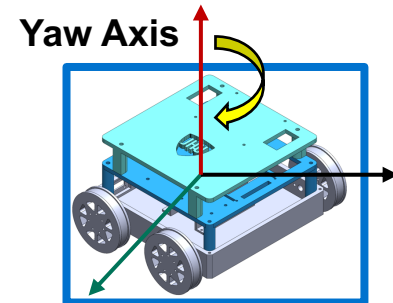
Algorithm 2 Yaw-Axis P Control Loop ($K_i = 0$, $K_d = 0$)

Require: Desired yaw rate ω_{ref} ; measured yaw rate g_z ; gain

K_p ; base PWM PWM_0

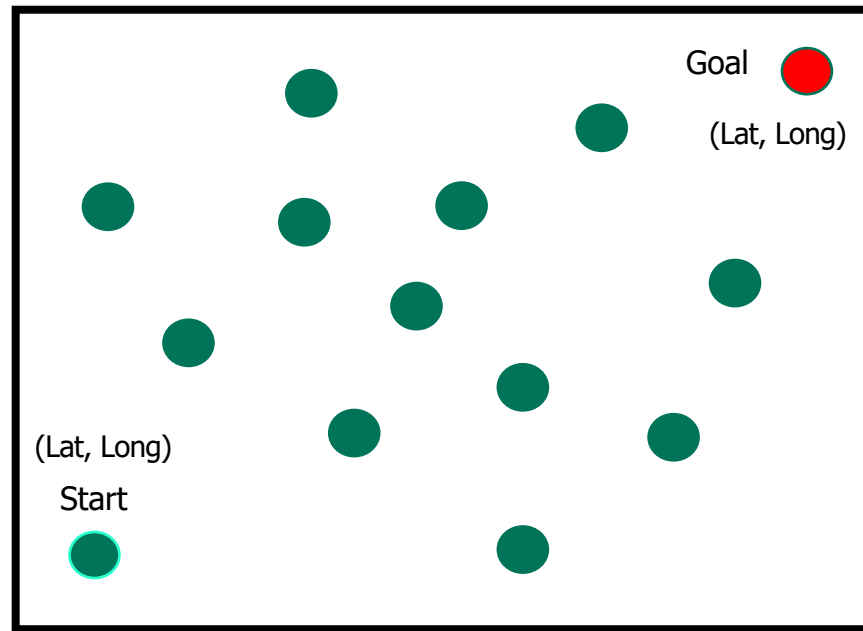
Ensure: Motor PWMs (PWM_{left} , PWM_{right})

```
1:  $e_{prev} \leftarrow 0$ 
2: while robot is active do
3:    $e \leftarrow \omega_{ref} - g_z$ 
4:    $u \leftarrow K_p \cdot e$   $\triangleright$  Since  $K_i = 0$  and  $K_d = 0$ , no I or D
    term
5:    $PWM_{left} \leftarrow PWM_0 - u$ 
6:    $PWM_{right} \leftarrow PWM_0 + u$ 
7:   Apply PWM via PCA9685
8:    $e_{prev} \leftarrow e$ 
9: end while
```



Example: Applications in Path & Motion Planning using $JH\mu$ BOT

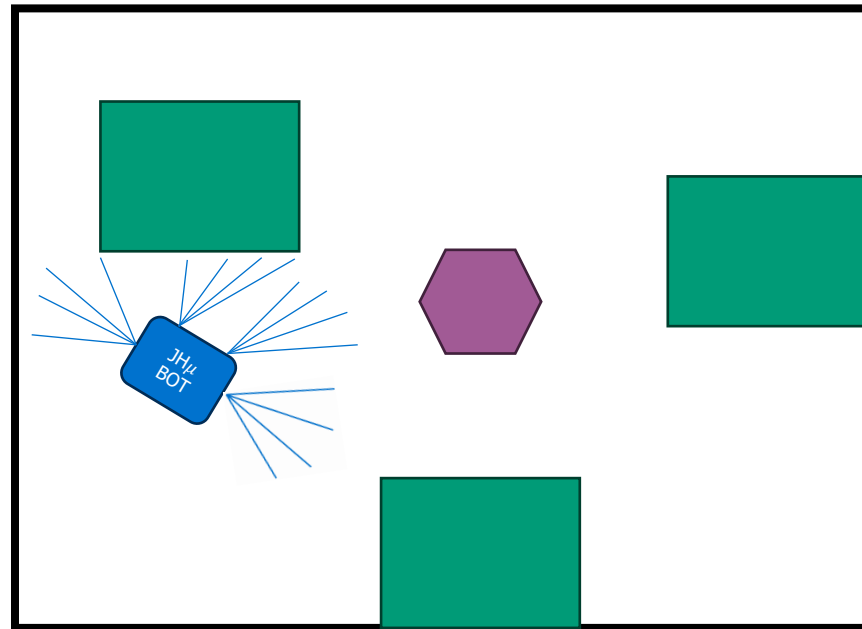
Rec area near JHU Campus



Equipped with GPS, LTE, and obstacle avoidance capabilities, the robot can receive target GPS coordinates from the user via MQTT or HTTP. Its position and the performance of its path and motion planning algorithms can be monitored in real time. Adding current sensors to each motor further enables analysis of the **power cost associated with different paths.**

Example: Applications in μ SLAM using JH μ BOT

With its IMU, odometry system, multi-channel ToF imaging sensor, and the dual-core architecture of the Raspberry Pi Pico, micro-SLAM techniques become feasible. Core 0 can manage sensor processing and robot control, while Core 1 handles data parsing and transmission. Using a local Python script on a desktop, real-time map creation is achievable.





JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2024, All Rights Reserved.